

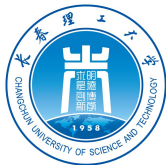
线段树与树状数组

简略讲解

wallcrack

计算机科学技术学院

2024年1月20日



- ① 引入
- ② 线段树
- ③ 树状数组
- ④ Thanks

- 1 引入
- 2 线段树
- 3 树状数组
- 4 Thanks

- 我们有时需要对序列或在值域上进行一些操作，例如区间求和。

- 我们有时需要对序列或在值域上进行一些操作，例如**区间求和**。
- 给定区间 $[l, r]$ 快速得到 $\sum_{i=l}^r a_i$ ，想必大家都已经能够轻松想出方法。预处理前缀和之后就能 $O(1)$ 得到区间和。

- 那么如果我们在区间询问之间插入一些修改操作，例如**单点修改**，即给定 x, v ，将 a_x 修改为 v ，此时预处理前缀和的方法就不再适用。

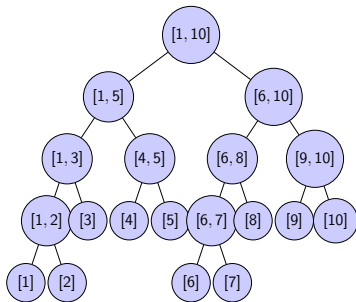
- 那么如果我们在区间询问之间插入一些修改操作，例如**单点修改**，即给定 x, v ，将 a_x 修改为 v ，此时预处理前缀和的方法就不再适用。
- 于是，我们就要用到特别的数据结构——**线段树或树状数组**，来完成这种类型任务。

- 那么如果我们在区间询问之间插入一些修改操作，例如**单点修改**，即给定 x, v ，将 a_x 修改为 v ，此时预处理前缀和的方法就不再适用。
- 于是，我们就要用到特别的数据结构——线段树或树状数组，来完成这种类型任务。
- 例题：P3374 【模板】树状数组

- 1 引入
- 2 线段树
- 3 树状数组
- 4 Thanks

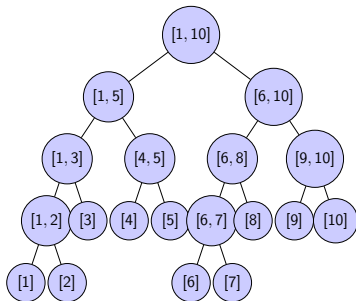
线段树示例

- 假设我们现在要维护一个长度为 10 的序列 A ，对应维护它的线段树结构就如右图。节点内标注的区间表明这个节点维护着这个区间的信息，在例题中就相当于维护着这个区间的区间和。



线段树示例

- 假设我们现在要维护一个长度为 10 的序列 A ，对应维护它的线段树结构就如右图。节点内标注的区间表明这个节点维护着这个区间的信息，在例题中就相当于维护着这个区间的区间和。
- 接下来，我们通过阅读代码来理解线段树的各种操作。



准备工作

```
1  const int N=2e5+10;
2  struct SegmentTree
3  {
4      #define lson (o<<1)
5      #define rson (o<<1|1)
6      #define mid (t[o].l+t[o].r>>1)
7      struct Node
8      {
9          int l,r,sumv;
10     }t[N<<2];
11 }
```

- 我们将数据结构封装起来，在外部尽量只调用接口函数，让代码更易读，让调试工作更简单。
- 每个节点的信息我们也打包起来，放在 Node 结构体中。

准备工作

```
1  const int N=2e5+10;
2  struct SegmentTree
3  {
4      #define lson (o<<1)
5      #define rson (o<<1|1)
6      #define mid (t[o].l+t[o].r>>1)
7      struct Node
8      {
9          int l,r,sumv;
10     }t[N<<2];
11 };
```

- 关于怎么在内存中实现这样的树状结构，假设该节点在数组中的位置为 o 那么其左儿子在数组中的位置为 $2o$ ，右儿子为 $2o+1$ 。如此我们便不需要在每个节点中都存储其左右儿子的位置的指针。

准备工作

```
1  const int N=2e5+10;
2  struct SegmentTree
3  {
4      #define lson (o<<1)
5      #define rson (o<<1|1)
6      #define mid (t[o].l+t[o].r>>1)
7      struct Node
8      {
9          int l,r,sumv;
10     }t[N<<2];
11 };
```

- 假设某节点维护着 $[l, r]$ 区间的信息，设 $mid = \lfloor \frac{l+r}{2} \rfloor$ ，那么左儿子则维护 $[l, mid]$ ，右儿子维护 $[mid + 1, r]$ 。

保持区间和性质

```
1  const int N=2e5+10;  
2  struct SegmentTree  
3  {  
4      void pushup(int o)  
5      {  
6          t[o].sumv=t[lson].  
              sumv+t[rson].  
              sumv;  
7      }  
8  };
```

- 当子节点数据被修改时，使用 `pushup` 就能合并两个区间的信息，维持节点的性质。

构造

```
1  const int N=2e5+10;
2  struct SegmentTree
3  {
4      void build(int o,int ql,int qr,
5                  int a[])
6      {
7          t[o].l=qr,t[o].r=qr;
8          if(ql==qr)
9          {
10             t[o].sumv=a[ql];
11             return ;
12          }
13          build(lson,ql,mid,a);
14          build(rson,mid+1,qr,a);
15          pushup(o);
16      };
```

- 每次给当前节点赋予其维护区间的左右端点，当进入到叶子节点时赋予其序列对应位置的值后返回。

修改

```
1  const int N=2e5+10;
2  struct SegmentTree
3  {
4      void modify(int o,int x,int v)
5      {
6          if(t[o].l==t[o].r)
7          {
8              t[o].sumv+=v;
9              return ;
10         }
11         if(mid<=x)modify(lson,x,v);
12         else modify(rson,x,v);
13         pushup(o);
14     }
15 };
```

- 每次判断待修改节点是在左儿子还是右儿子，之后进入对应节点。当区间长度为 1 时说明已经到达待修改节点，修改值后返回。

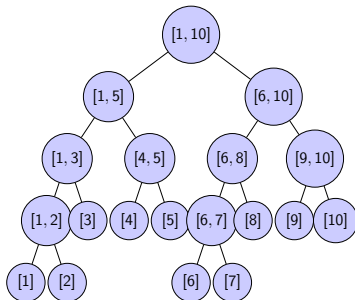
询问

```
1  const int N=2e5+10;
2  struct SegmentTree
3  {
4      int sum(int o,int ql,int qr)
5      {
6          if(t[o].l>=ql && t[o].r<=qr)
7              return t[o].sumv;
8          int res=0;
9          if(ql<=mid)
10             res+=sum(lson,ql,qr);
11             if(qr>mid)
12                 res+=sum(rson,ql,qr);
13             return res;
14     }
15 };
```

- 若当前节点维护区间被询问区间包含了就直接返回当前节点的区间和，否则分别判断其左右儿子与询问区间是否有交集，有交集就递归询问。

练习时间

- 借助代码，完成例题。



- ① 引入
- ② 线段树
- ③ 树状数组
- ④ Thanks

lowbit操作

- $\text{lowbit}(x)$ 定义为最小的满足 $2^y \& x \neq 0$ 的 2^y 。

结构

- 在树状数组中，数组的第 x 位就维护着 $[x - \text{lowbit}(x) + 1, x]$ 这个区间的前缀和。

总体代码

```
1 struct BIT
2 {
3     long long siz,c[N];
4     void build(int n)
5     {
6         siz=n;
7     }
8     int lowbit(int x)
9     {
10         return x&(-x);
11     }
12     void add(int x,long long v)
13     {
14         for(int i=x;i<=siz;i+=lowbit(i))
15             c[i]+=v;
16     }
17     long long sum(int l,int r)
18     {
19         long long sumv=0;
20         for(int i=r;i>=1;i-=lowbit(i))
21             sumv+=c[i];
22         for(int i=l-1;i>=1;i-=lowbit(i))
23             sumv-=c[i];
24         return sumv;
25     }
26 };
```

- ① 引入
- ② 线段树
- ③ 树状数组
- ④ Thanks

Thanks!